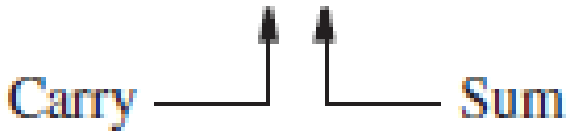
ARITHMETIC CIRCUITS

Addition of Unsigned Numbers

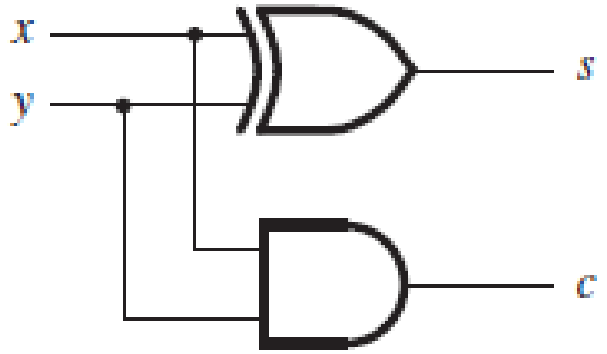
The one-bit addition entails four possible combinations,

x	0	0	1	1
$+\ y$	$+\ 0$	$+\ 1$	$+\ 0$	$+\ 1$
$c\ s$	0 0	0 1	0 1	1 0



		Carry	Sum
x	y	c	s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

(b) Truth table



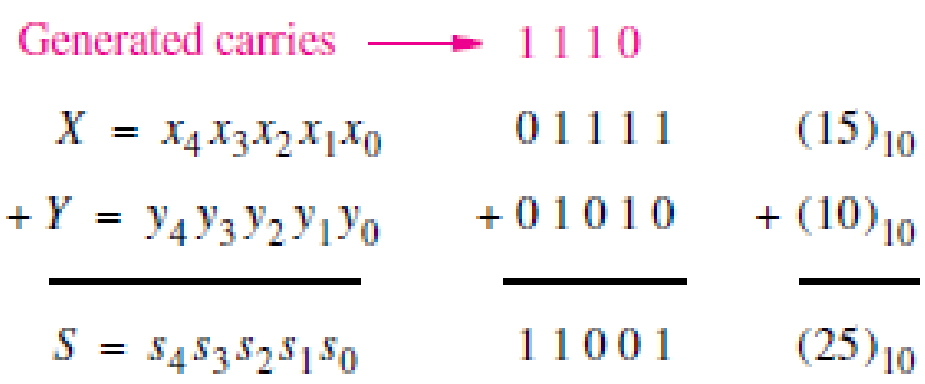
(c) Circuit



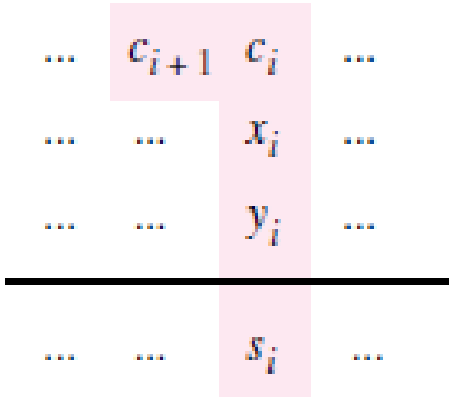
(d) Graphical symbol

The sum bit *s* is the XOR function. The carry *c* is the AND function of inputs *x* and *y*. A circuit realization of these functions is shown in Fig. *c*. This circuit, which implements the addition of only two bits, is called a *half-adder*.

Multibit Addition



(a) An example of addition



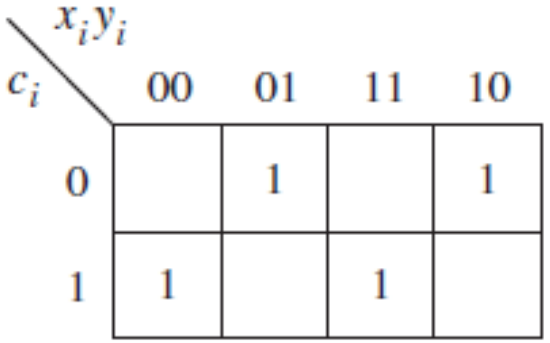
(b) Bit position i

for each bit position i, the addition operation may include a *carry-in* from bit position $i - 1$.

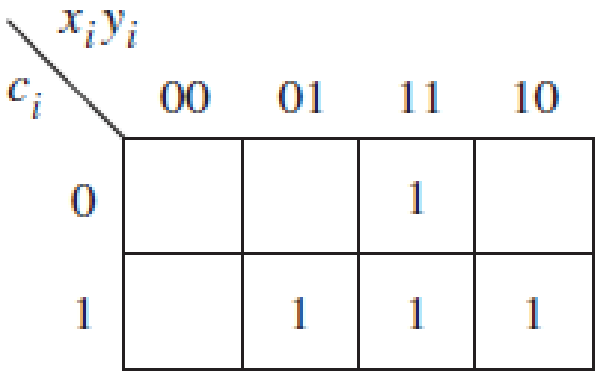
This observation leads to the design of a logic circuit that has three inputs *xi* , *yi* , and *ci* , and produces the two outputs *si* and *ci*+1.

c_i	x_i	y_i	c_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

(a) Truth table



$$s_i = x_i \oplus y_i \oplus c_i$$



$$c_{i+1} = x_iy_i + x_ic_i + y_ic_i$$

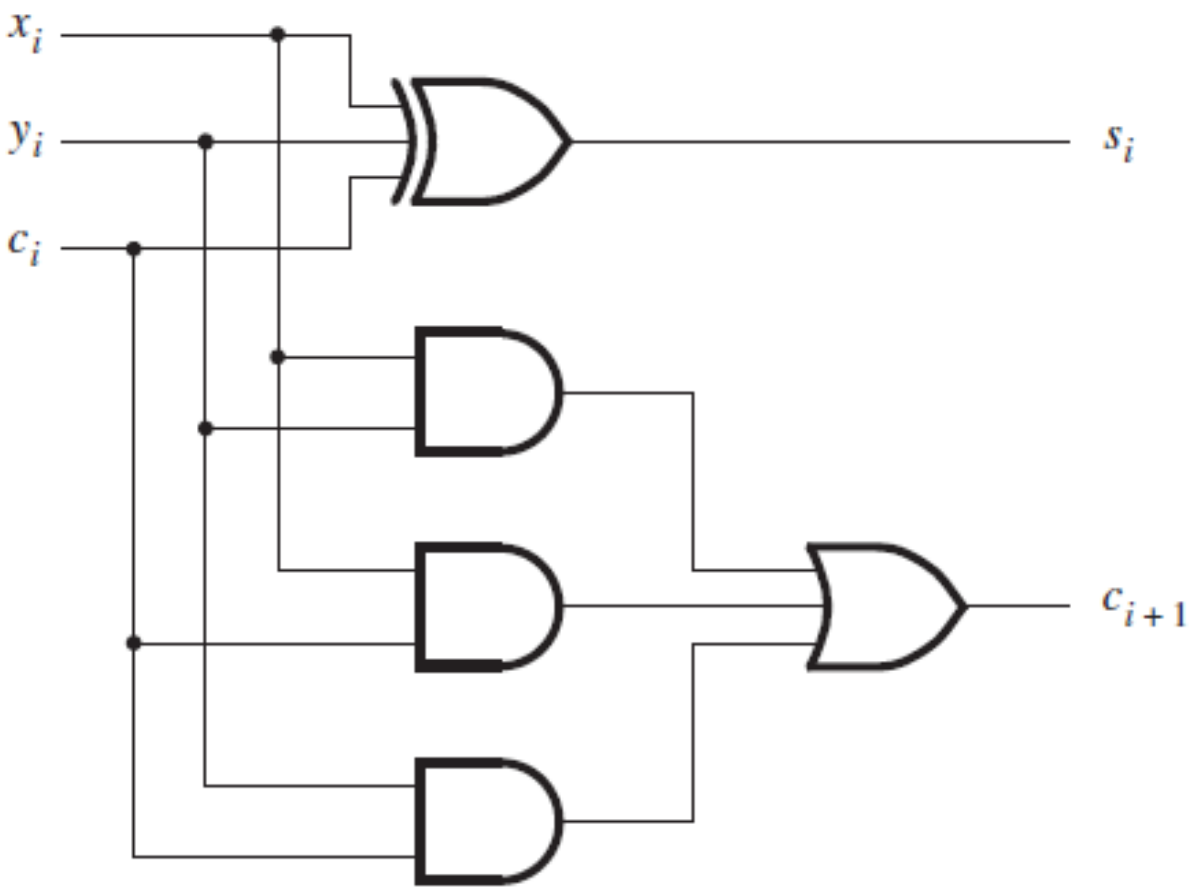
$$s_i = \overline{x}_i y_i \overline{c}_i + x_i \overline{y}_i \overline{c}_i + \overline{x}_i \overline{y}_i c_i + x_i y_i c_i$$

$$s_i = (\overline{x}_i y_i + x_i \overline{y}_i)\overline{c}_i + (\overline{x}_i \overline{y}_i + x_i y_i)c_i$$

$$= (x_i \oplus y_i)\overline{c}_i + \overline{(x_i \oplus y_i)}c_i$$

$$= (x_i \oplus y_i) \oplus c_i$$

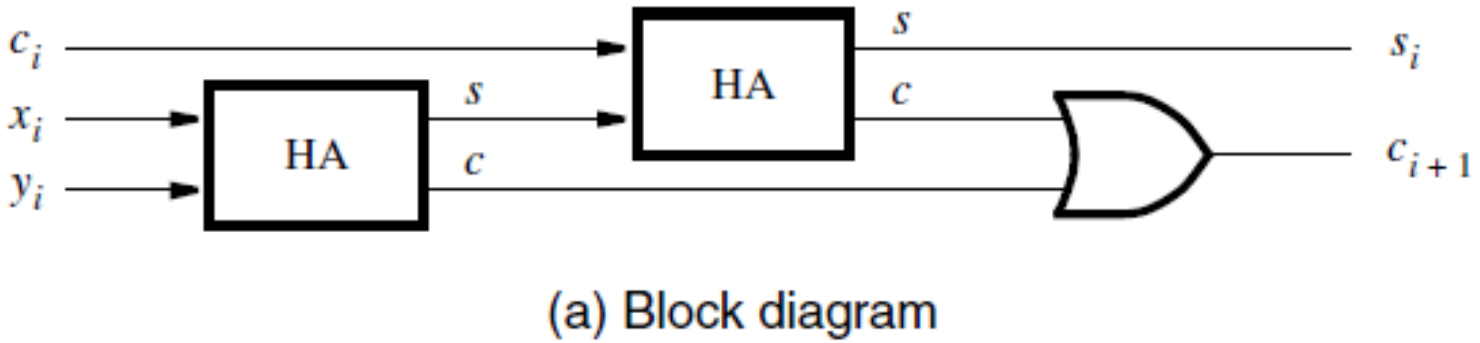
$$s_i = x_i \oplus y_i \oplus c_i$$



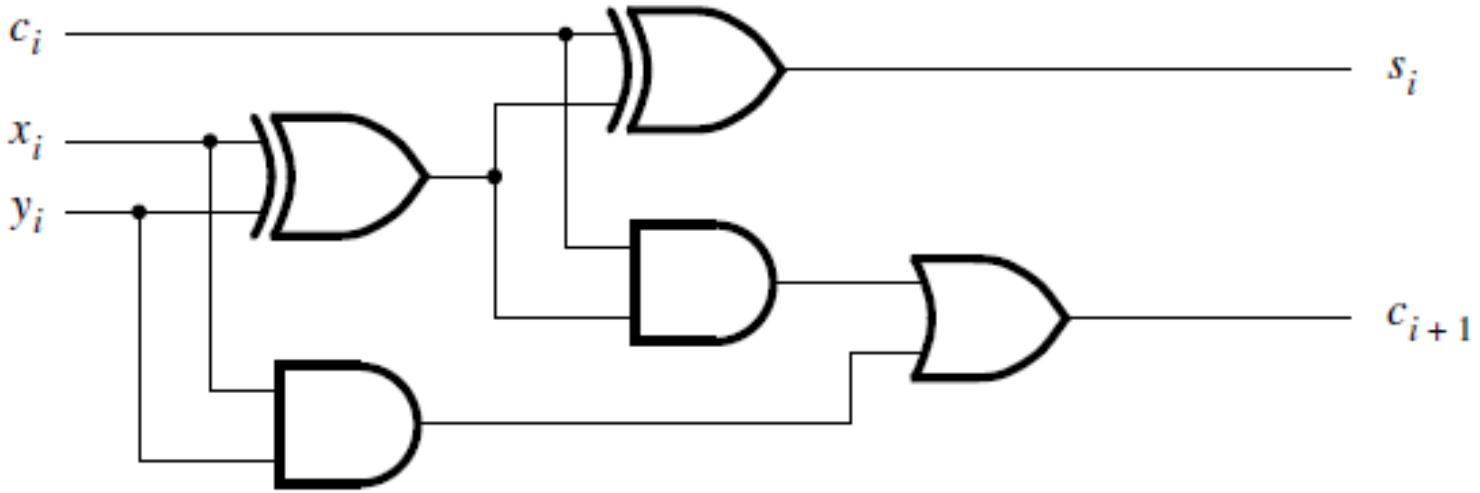
(c) Circuit

This circuit is known as a *full-adder*.

Decomposed Full-Adder



(a) Block diagram



(b) Detailed diagram

S1=xi XOR yi

C1=xiyi

S2=Si=S1 XOR ci = xi XOR yi XOR ci

C2=S1ci = (xi XOR yi)ci

= xi’yici + xiyi’ci

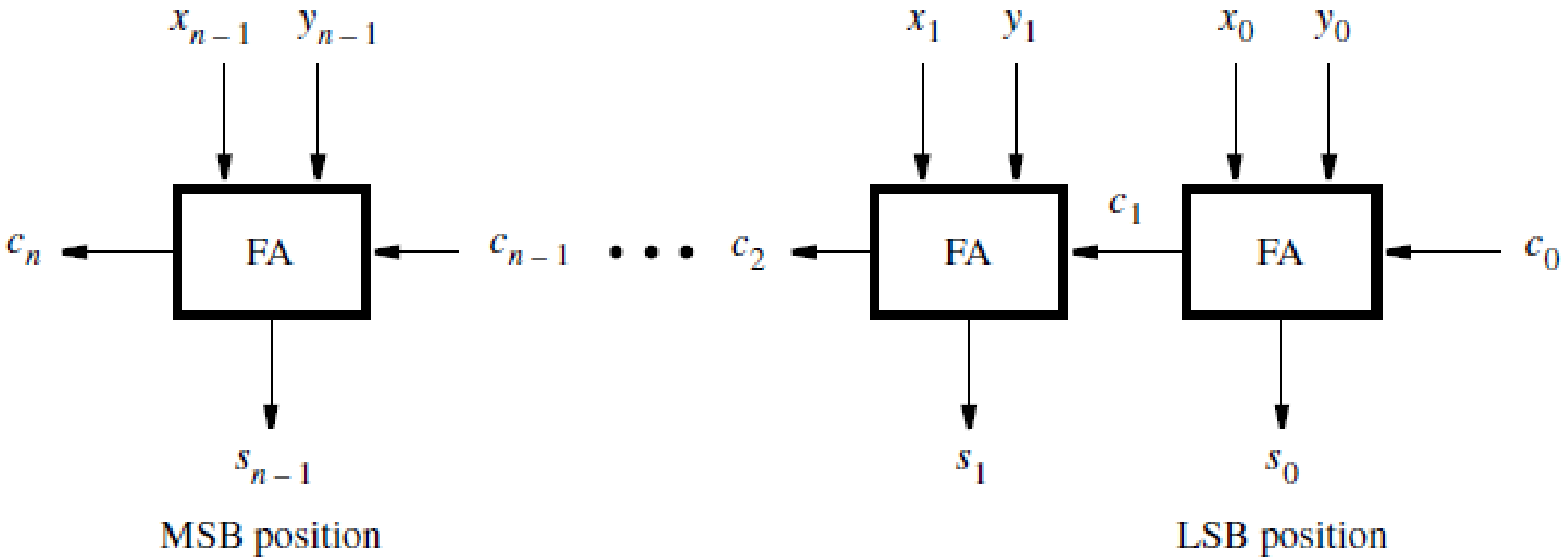
Ci+1= xiyi + xi’yici + xiyi’ci

= xiyi(ci+ci’) + xi’yici + xiyi’ci

= xiyici +xiyici’ + xi’yici + xiyi’ci

=xiyi +yici + xici

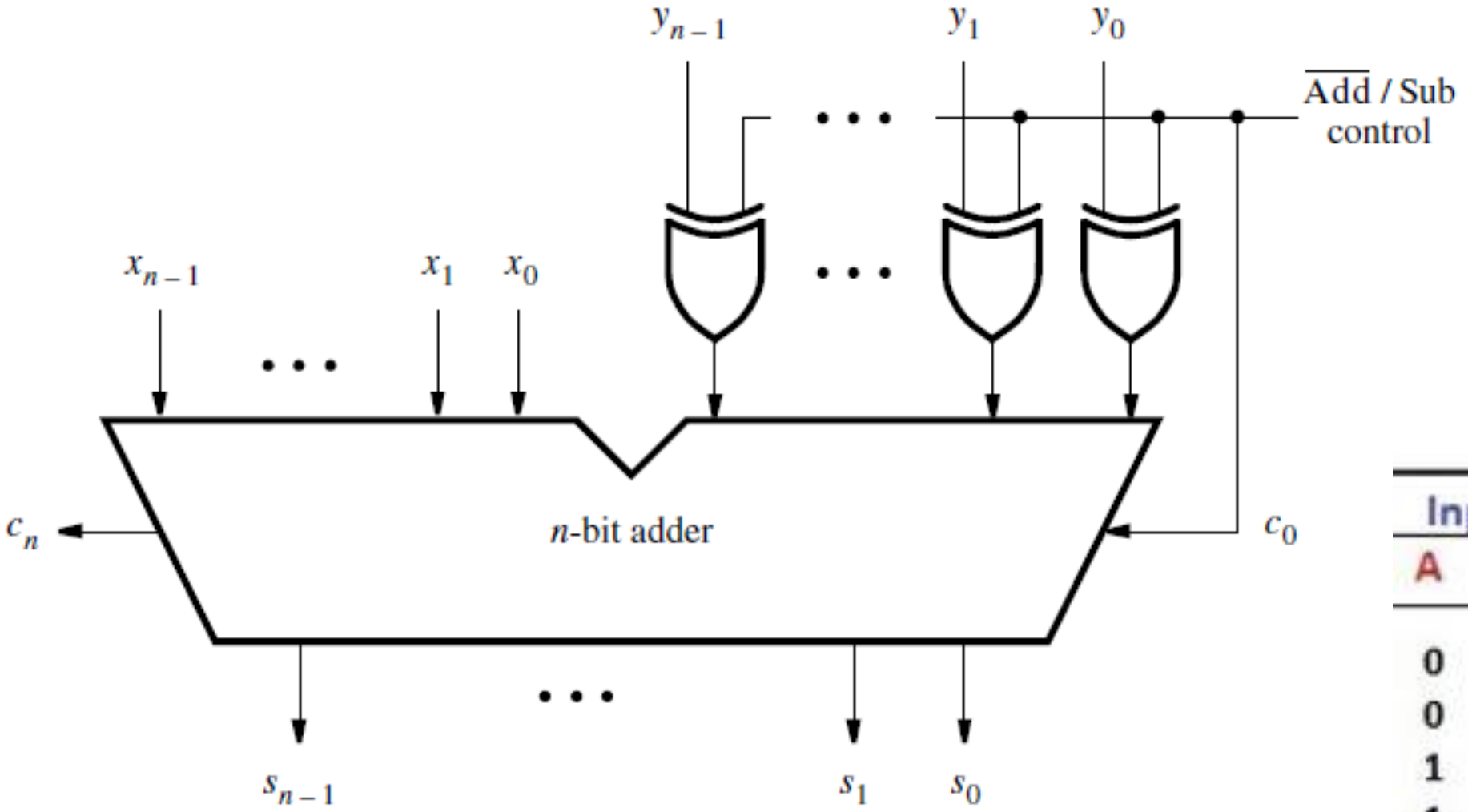
Ripple-Carry Adder



For each bit position we can use a full-adder circuit, connected as shown in Figure

- When the operands X and Y are applied as inputs to the adder, it takes some time before the output sum, S, is valid. Each full-adder introduces a certain delay before its si and $ci+1$ outputs are valid. Let this delay be denoted as Δt.
- Thus the carry-out from the first stage, $c1$, arrives at the second stage Δt after the application of the $x0$ and $y0$ inputs. The carry-out from the second stage, $c2$, arrives at the third stage with a $2\Delta t$ delay, and so on.
- The signal $cn-1$ is valid after a delay of $(n - 1)\ \Delta t$, which means that the complete sum is available after a delay of $n\Delta t$. Because of the way the carry signals "ripple" through the full-adder stages, the circuit is called a ripple-carry adder.

Adder and Subtractor Unit



Inputs		Output
A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

When Add/Sub = 0, it performs **X + Y**.

When Add/Sub = 1, it performs

X + Y’ + 1

= **X + 2’s complement of Y**

= **X - Y**

Arithmetic Overflow

The result of addition or subtraction is supposed to fit within the significant bits used to represent the numbers. If n bits are used to represent signed numbers, then the result must be in the range -2^{n-1} to $2^{n-1} - 1$. If the result does not fit in this range, then we say that *arithmetic overflow* has occurred.

$$
\begin{array}{r}
(+7) \\
+\,(+2) \\
\hline
(+9)
\end{array}
\qquad
\begin{array}{r}
0\,1\,1\,1 \\
+\;0\,0\,1\,0 \\
\hline
1\,0\,0\,1
\end{array}
$$

$c_4 = 0$
$c_3 = 1$

$$
\begin{array}{r}
(-7) \\
+\,(+2) \\
\hline
(-5)
\end{array}
\qquad
\begin{array}{r}
1\,0\,0\,1 \\
+\;0\,0\,1\,0 \\
\hline
1\,0\,1\,1
\end{array}
$$

$c_4 = 0$
$c_3 = 0$

$$
\begin{array}{r}
(+7) \\
+\,(-2) \\
\hline
(+5)
\end{array}
\qquad
\begin{array}{r}
0\,1\,1\,1 \\
+\;1\,1\,1\,0 \\
\hline
1\,0\,1\,0\,1
\end{array}
$$

$c_4 = 1$
$c_3 = 1$

$$
\begin{array}{r}
(-7) \\
+\,(-2) \\
\hline
(-9)
\end{array}
\qquad
\begin{array}{r}
1\,0\,0\,1 \\
+\;1\,1\,1\,0 \\
\hline
1\,0\,1\,1\,1
\end{array}
$$

$c_4 = 1$
$c_3 = 0$

We are using four-bit numbers. If both numbers have the same sign, the magnitude of the result is 9, which cannot be represented with 4 bits in 2's complement representation. Therefore, overflow occurs. When the numbers have opposite signs, there is no overflow. The key to determine whether overflow occurs is the carry-out from the sign-bit position, called c_4 in the figure and from the previous bit position, called c_3. The figure indicates that overflow occurs when these carry-outs have different values, and a correct sum is produced when they have the same value. Indeed, this is true in general for both addition and subtraction of 2's-complement numbers.

$$\begin{aligned}\text{Overflow} &= c_3\overline{c}_4 + \overline{c}_3 c_4 \\ &= c_3 \oplus c_4\end{aligned}$$

For n-bit numbers we have

$$\text{Overflow} = c_{n-1} \oplus c_n$$

An alternative and more intuitive way of detecting the arithmetic overflow is to observe that overflow occurs if both summands have the same sign but the resulting sum has a different sign. Let $X = x3x2x1x0$ and $Y = y3y2y1y0$ represent four-bit 2's-complement numbers, and let $S = s3s2s1s0$ be the sum $S = X + Y$. Then

$$\text{Overflow} = x_3 y_3 \overline{s}_3 + \overline{x}_3 \overline{y}_3 s_3$$

The carry-out and overflow signals indicate whether the result of a given addition is too large to fit into the number of bits available to represent the sum. The carry-out is meaningful only when unsigned numbers are involved, while the overflow is meaningful only in the case of signed numbers.

Fast Adders

The addition and subtraction of numbers are fundamental operations that are performed frequently in the course of a computation. The speed with which these operations are performed has a strong impact on the overall performance of a computer.

Let us take a closer look at the speed of the adder/subtractor unit. We are interested in the largest delay from the time the operands X and Y are presented as inputs, until the time all bits of the sum S and the final carry-out, c_n, are valid.

The delay for the carry-out signal in the full adder circuit, Δt, is equal to two gate delays.
The final result of the addition will be valid after a delay of $n\Delta t$, which is equal to 2n gate delays.

In addition to the delay in the ripple-carry path, there is also a delay in the XOR gates that feed either the true or complemented value of Y to the adder inputs. If this delay is equal to one gate delay, then the total delay of the circuit is $2n + 1$ gate delays. For a large n, say $n = 32$ or $n = 64$, the delay would lead to unacceptably poor performance.

The longest delay is along the path from the yi input, through the XOR gate and through the carry circuit of each adder stage. The longest delay is often referred to as the *critical-path delay*, and the path that causes this
delay is called the *critical path*.

Carry-Lookahead Adder

To reduce the delay caused by the effect of carry propagation through the ripple-carry adder, we can attempt to evaluate quickly for each stage whether the carry-in from the previous stage will have a value 0 or 1. If a correct evaluation can be made in a relatively short time, then the performance of the complete adder will be improved.

$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$

$$c_{i+1} = x_i y_i + (x_i + y_i) c_i$$

$$c_{i+1} = g_i + p_i c_i$$

$$g_i = x_i y_i$$

$$p_i = x_i + y_i$$

g_i is called the *generate* function.
p_i is called the *propagate* function.

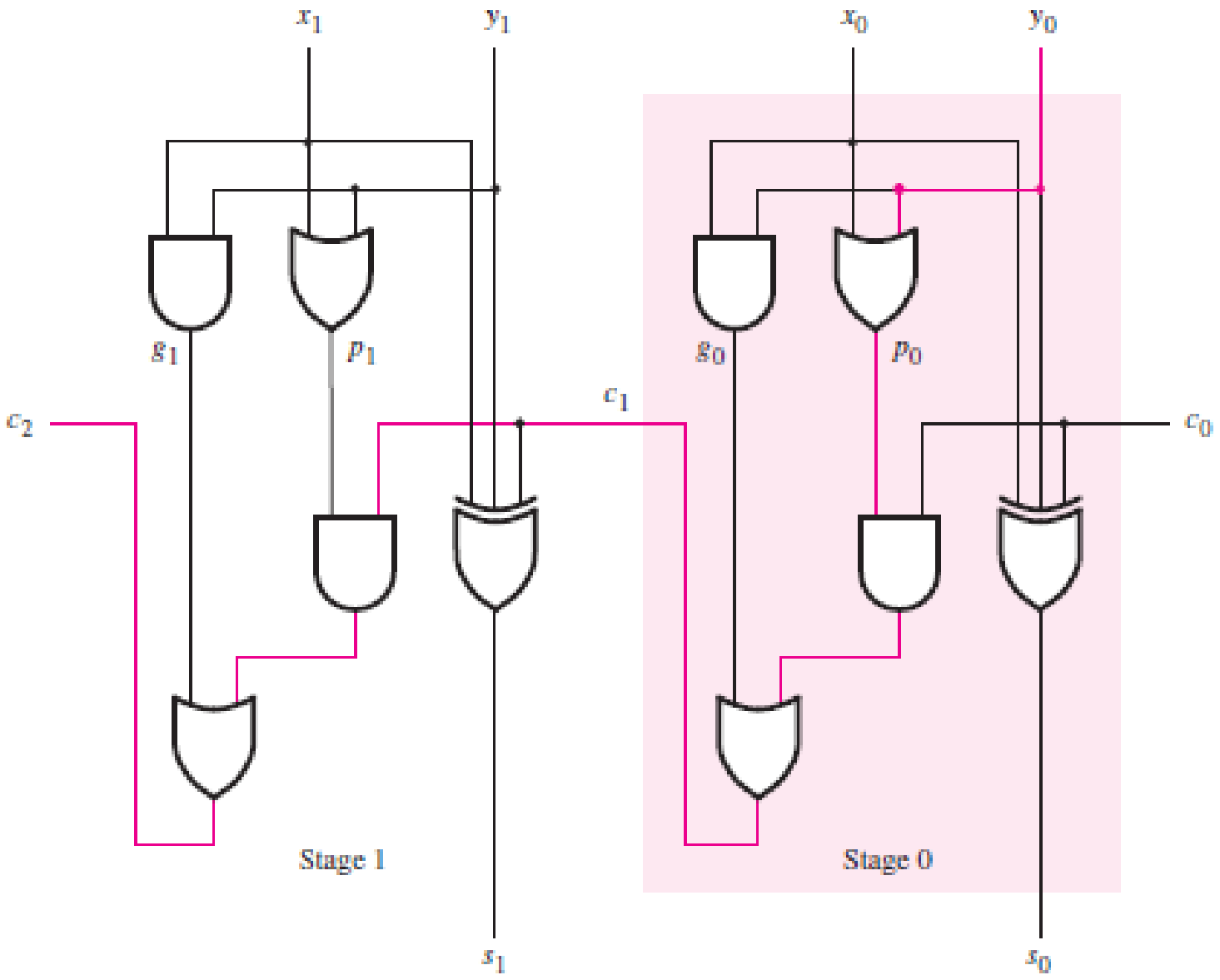
Expanding in terms of stage $i - 1$ gives

$$\begin{aligned} c_{i+1} &= g_i + p_i(g_{i-1} + p_{i-1}c_{i-1}) \\ &= g_i + p_i g_{i-1} + p_i p_{i-1} c_{i-1} \end{aligned}$$

The same expansion for other stages, ending with stage 0, gives

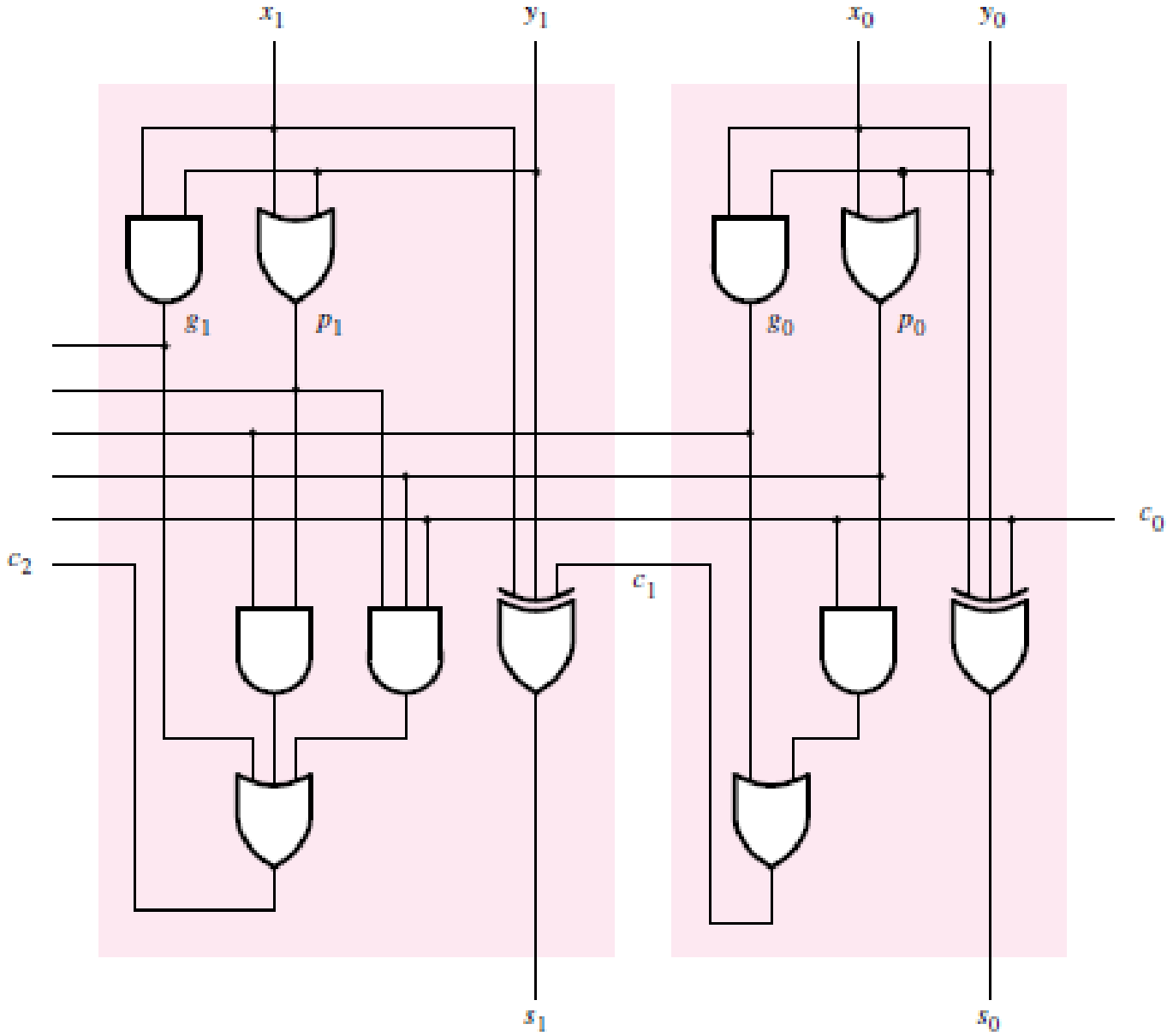
$$c_{i+1} = g_i + p_i g_{i-1} + p_i p_{i-1} g_{i-2} + \cdots + p_i p_{i-1} \cdots p_2 p_1 g_0 + p_i p_{i-1} \cdots p_1 p_0 c_0$$

This expression represents a two-level AND-OR circuit in which c_{i+1} is evaluated very quickly. An adder based on this expression is called a *carry-lookahead adder*.



A ripple-carry adder

The critical path is from inputs $x0$ and $y0$ to the output $c2$. It passes through five gates, The path in other stages of an n-bit adder is the same as in stage 1. Therefore, the total number of gate delays along the critical path is $2n + 1$.

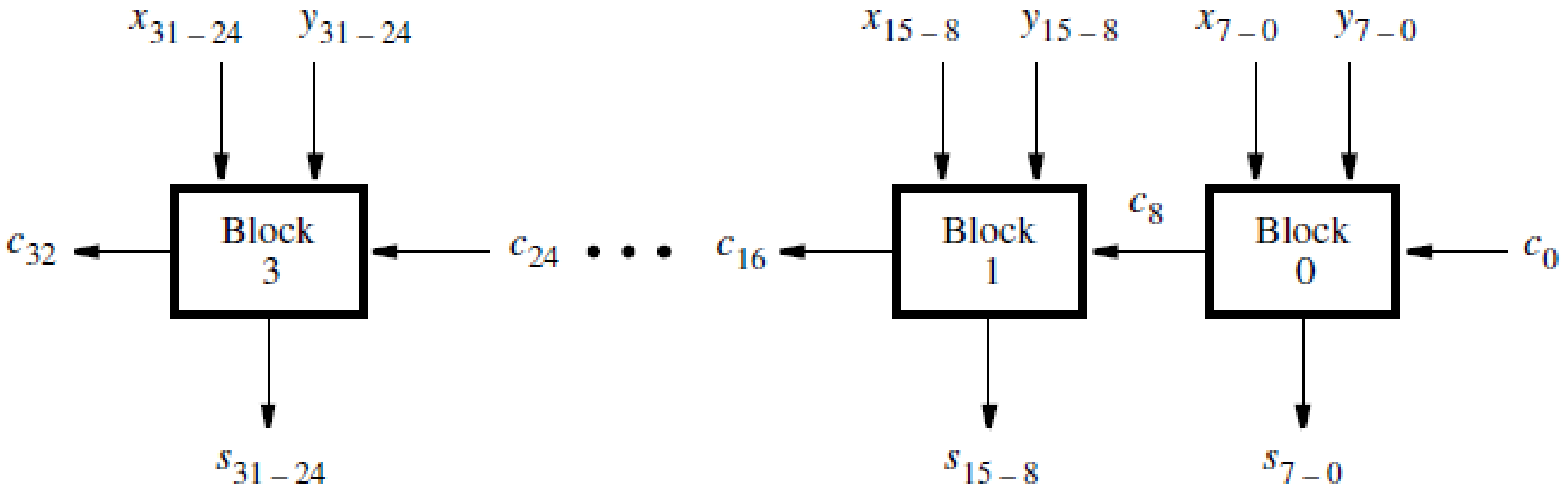


The first two stages of a carry-lookahead adder

All carry signals are produced after three gate delays: one gate delay is needed to produce the generate and propagate signals $g0$, $g1$, $p0$, and $p1$, and two more gate delays are needed to produce $c1$ and $c2$ concurrently. Extending the circuit to n bits, the final carry-out signal cn would also be produced after only three gate delays.

The total delay in the *n*-bit carry-lookahead adder is four gate delays. The values of all *gi* and *pi* signals are determined after one gate delay. It takes two more gate delays to evaluate all carry signals. Finally, it takes one more gate delay (XOR) to generate all sum bits. The key to the good performance of the adder is quick evaluation of carry signals.

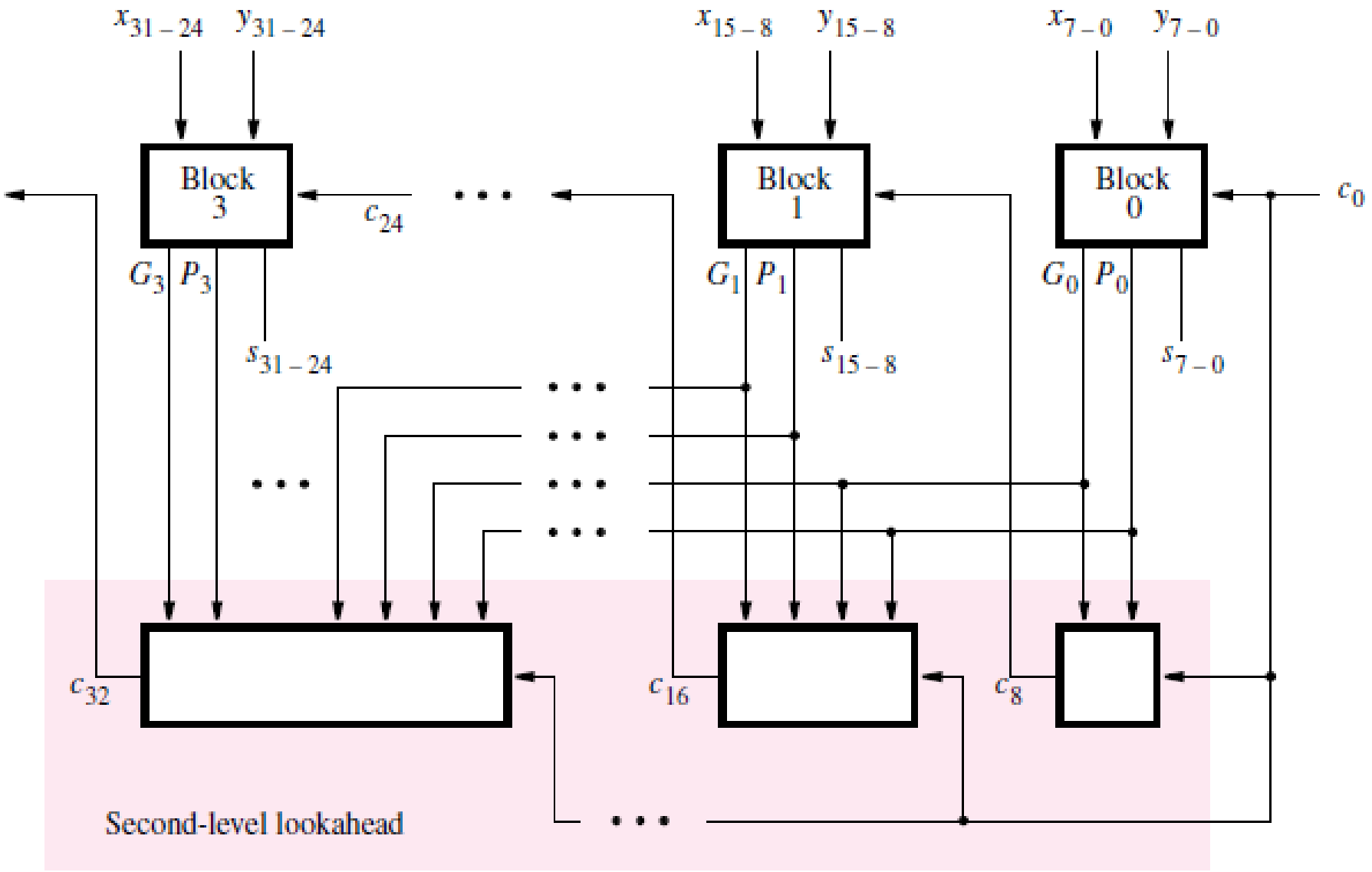
- The complexity of an *n*-bit carry-lookahead adder increases rapidly as *n* becomes larger.
- To reduce the complexity, we can use a *hierarchical* approach in designing large adders.
- Suppose that we want to design a 32-bit adder. We can divide this adder into 4 eight-bit blocks, such that block 0 adds bits 7 . . . 0, block 1 adds bits 15 . . . 8, block 2 adds bits 23 . . . 16, and block 3 adds bits 31 . . . 24. Then we can implement each block as an eight-bit carry-lookahead adder. The carry-out signals from the four blocks are *c*8, *c*16, *c*24, and *c*32. Now we have two possibilities.
- We can connect the four blocks as four stages in a ripple-carry adder. Thus while carry-lookahead is used within each block, the carries ripple between the blocks.



A hierarchical carry-lookahead adder with ripple-carry between blocks.

gi, pi : 1 gate delay
c1-c8 : 2 gate delay
c9-c16 : 2 gate delay
c17-c24 : 2 gate delay
c25-c32 : 2 gate delay
One more gate delay to produce the sum bits form the last block
Total : 10 gate dealy.

Instead of using a ripple-carry approach between blocks, a faster circuit can be designed in which a second-level carry-lookahead is performed to produce quickly the carry signals between blocks. The structure of this “hierarchical carry-lookahead adder” is shown in the figure below



A hierarchical carry-lookahead adder.

- Each block in the top row includes an eight-bit carry-lookahead adder, based on generate and propagate signals for each stage in the block.
- Instead of producing a carry-out signal from the most-significant bit of the block, each block produces generate and propagate signals for the entire block. Let *Gj* and *Pj* denote these signals for each block *j*.
- Now *Gj* and *Pj* can be used as inputs to a second-level carrylookahead circuit at the bottom of the Figure, which evaluates all carries between blocks.
- We can derive the block generate and propagate signals for block 0 by examining the expression for *c*8

$$c_8 = g_7 + p_7g_6 + p_7p_6g_5 + p_7p_6p_5g_4 + p_7p_6p_5p_4g_3 + p_7p_6p_5p_4p_3g_2 + p_7p_6p_5p_4p_3p_2g_1 + p_7p_6p_5p_4p_3p_2p_1g_0 + p_7p_6p_5p_4p_3p_2p_1p_0c_0$$

$$P_0 = p_7p_6p_5p_4p_3p_2p_1p_0$$

$$G_0 = g_7 + p_7g_6 + p_7p_6g_5 + \cdots + p_7p_6p_5p_4p_3p_2p_1g_0$$

$$c_8 = G_0 + P_0c_0$$

For block 1 the expressions for $G1$ and $P1$ have the same form as for $G0$ and $P0$ except that each subscript i is replaced by $i + 8$. The expressions for $G2$, $P2$, $G3$, and $P3$ are derived in the same way. The expression for the carry-out of block 1, c16, is

$$c_{16} = G_1 + P_1 c_8$$
$$= G_1 + P_1 G_0 + P_1 P_0 c_0$$

Similarly, the expressions for $c24$ and $c32$ are

$$c_{24} = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 c_0$$
$$c_{32} = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 c_0$$

Using this scheme, it takes two more gate delays to produce the carry signals $c8$, $c16$, $c24$, and $c32$ than the time needed to generate the Gj and Pj functions. Therefore, since Gj and Pj require three gate delays, $c8$, $c16$, $c24$, and $c32$ are available after five gate delays. The time needed to add two 32-bit numbers involves these five gate delays plus two more to produce the internal carries in blocks 1, 2, and 3, plus one more gate delay (XOR) to generate each sum bit. This gives a total of eight gate delays.

It takes $2n + 1$ gate delays to add two numbers using a ripple-carry adder. For 32-bit numbers this implies 65 gate delays. It is clear that the carry-lookahead adder offers a large performance improvement.